

Compiler Construction

⋈ Basic Blocks ⋈

Problem statement (1/2)

How to translate two way conditional jumps from HIR to LIR?

Problem statement (2/2)

This does not exist in ASM

```
cjump xxx  $\alpha$   $\beta$ 
```

Since cjump (in assembly language) has a label, rather than an expression AND only one label...

```
cjump xxx name Label_alpha  
 $\beta$   
...  
label Label_alpha  
 $\alpha$ 
```

Two Way Jumps

Obviously, to enable the translation of a *cjump* into actual assembly instructions, the “false” label must follow the *cjump*.

How?

Basic Blocks

We must analyze the flow of the program

Basic Block

A basic block is a sequence of statements that is always entered at the beginning and exited at the end.

- The first statement is a Label
- The last statement is a jump or a cjump
- There are no other Labels, jumps or cjump in the block

Algorithm for building Basic Blocks

- ➊ Scan the sequence from the beginning to the end
- ➋ When a label is found, start a new block (and end the previous block)
- ➌ Whenever a cjump/jump is found the current block is ended (and the next block is started)
- ➍ If this leaves a block ending without a cjump/jump, then append a jump to the next block
- ➎ If a block has no Label at the beginning, invent one, and add it

Exercise - 1

Convert this code to basic blocks

```
label prologue
    Prologue.
label test
cjump le i N body done
label body
    Body.
jump name test
label done
    Epilogue
jump name end
```

Exercise - 1

Cut at labels, and possibly insert jumps.

```
label prologue
    Prologue.
jump name test
```

```
label test
cjump le i N body done
label body
    Body.
jump name test
label done
    Epilogue
jump name end
```


Exercise - 1

Cut at jumps.

```
label prologue  
    Prologue.  
jump name test
```

```
label test  
cjump le i N body done
```

```
label body  
    Body.  
jump name test  
label done  
    Epilogue  
jump name end
```

Exercise - 1

```
label prologue  
    Prologue.  
jump name test
```

```
label test  
cjump le i N body done
```

```
label body  
    Body.  
jump name test
```

```
label done  
    Epilogue  
jump name end
```

Traces and Control-Flow Graphs

Control-Flow Graph

A program is represented as a graph:

- Nodes are **basic blocks**
- Edges represent possible transfers of control

Goal

Reorder basic blocks *without changing semantics*, while improving fall-throughs and cache behavior.

What is a Trace?

Trace

A **trace** is a simple path in the control-flow graph:

- A sequence of one or more basic blocks
- Each block transfers control to the next one
- No block appears more than once

Key idea

A trace represents a sequence of blocks that *may execute consecutively* at runtime.

Why Traces Matter

Code layout

Ideally, the ordering of basic blocks should:

- Favor **fall-through execution**
- Group **frequently executed blocks**

Instruction cache

Keeping hot code inside a trace improves locality and reduces taken branches.

Greedy Trace Construction

Greedy idea

Build traces incrementally, choosing the next block as long as control-flow allows it.

- ① Mark all basic blocks as **unmarked**
- ② While unmarked blocks remain:
 - ▶ Pick one unmarked block (heuristic choice)
 - ▶ Extend a trace by following edges to unmarked blocks
 - ▶ Mark each visited block

Important

Different orderings are correct, but not equally efficient.

CJUMPs After Traces

Canonical form

After trace construction:

- The **false** branch of a cjump should fall through
- Each cjump has exactly one explicit label

Result

Two-way conditional jumps are now directly translatable to assembly.

Algorithm to build Traces (1/2)

Main Idea

Start from the initial block, and “sew” each remaining basic block to this growing “trace”.

Algorithm to build Traces (2/2)

- ❶ If the last instruction is a *jump*
 - ▶ if the “destination block” is available, add it
 - ▶ otherwise, fetch any other remaining block.

- ❷ If the last instruction is a *cjump*
 - ▶ If the false destination is available, push it
 - ▶ If the true destination is available, flip the *cjump* and push it,
 - ▶ otherwise, change the *cjump* to go to a fresh label, attach this label, and finally *jump* to the initial false destination.

Two Way Jumps: Optimizing Traces

Two Way Jumps: Optimizing Traces

```
label prologue  
  Prologue  
jump name test
```

```
label test  
cjump gt i N  
  done body
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

Two Way Jumps: Optimizing Traces

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label test  
cjump gt i N  
  done body
```

```
label test  
cjump le i N  
  body done
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

```
label done  
  Epilogue  
jump name end
```

```
label body  
  Body  
jump name test
```

Two Way Jumps: Optimizing Traces

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label test  
cjump gt i N  
  done body
```

```
label test  
cjump le i N  
  body done
```

```
label body  
  Body  
jump name test
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

```
label test  
cjump le i N  
  body done
```

```
label done  
  Epilogue  
jump name end
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

Hot loops

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label test  
cjump gt i N  
  done body
```

```
label test  
cjump le i N  
  body done
```

```
label body  
  Body  
jump name test
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

```
label test  
cjump le i N  
  body done
```

```
label done  
  Epilogue  
jump name end
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

Hot loops

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label prologue  
  Prologue  
jump name test
```

```
label test  
cjump gt i N  
  done body
```

```
label test  
cjump le i N  
  body done
```

```
label body  
  Body  
jump name test
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

```
label test  
cjump le i N  
  body done
```

```
label done  
  Epilogue  
jump name end
```

```
label body  
  Body  
jump name test
```

```
label done  
  Epilogue  
jump name end
```

Summary

